Project Name: AI Code Navigator (Backend) (week - 7)

What is the idea behind this project?

Imagine you join a new company and they give you a big codebase with hundreds of files. You have no idea what is written where. Normally you would spend days reading each file one by oneThis project solves that problem. You just give it the GitHub link, it reads everything automatically, and then you can simply ask "how does the login work?" or "where is the payment code?" and it will tell you in plain English.

So basically this project is like a smart assistant for understanding code.

How does the server work?

We used Node.js and Express to build the server. When you start the server it does two things at the same time. First it connects to MongoDB database where all the code and summaries will be stored. Second it starts listening on port 5000 for any requests coming from the frontend or from tools like Postman.

The server has 4 routes which means 4 things you can do with it.

Feature 1 - Upload a GitHub Repo

This is the main feature. You send a POST request to /api/upload-repo with a GitHub link like https://github.com/username/reponame.

What happens behind the scenes:

- The server takes the link and figures out the owner name and repo name from it
- Then it goes to GitHub and downloads the entire repo as a ZIP file
- The ZIP file gets saved temporarily in the system's temp folder
- Then it opens the ZIP and goes through every single file inside
- It skips useless files like images, videos, or font files
- For every code file like .js, .py, .html, .css etc it reads the code inside
- It sends that code to OpenAI and says "summarize this"
- OpenAI reads it and gives back a short summary in plain English

- The file name, the actual code, and the summary all get saved together in MongoDB
- At the end the ZIP file is deleted from the temp folder to save space
- The server then sends back a response saying how many files were processed

We tested this on your petadoption repo and it successfully processed all the React files including components, pages, routes, Firebase config, and Tailwind config.

Feature 2 - Upload Files from Computer

This is for when you do not want to use a GitHub link. You can just pick files from your computer and upload them directly to /api/upload. You can upload up to 10 files at once.

We used a tool called Multer which handles file uploads in Node.js. It temporarily saves the uploaded files in an uploads/ folder. Then the server reads each file, sends it to OpenAI for a summary, saves everything in MongoDB, and then deletes the temporary file from the uploads folder.

Feature 3 - Process a Single Code Snippet

This is the simplest one. You send a POST request to /api/process-code with just raw code text in the request body along with a file name. The server sends it to OpenAI, gets the summary back, saves it in MongoDB, and returns the summary to you. This is useful if you just want to quickly understand one specific piece of code without uploading a whole project.

Feature 4 - Ask Questions About the Code

This is the coolest part. Once your code is stored in MongoDB, you can go to /api/search and ask any question you want in plain English.

What happens:

- The server fetches every single code document from MongoDB
- It combines all the code into one big text
- It adds your question at the end
- It sends the whole thing to OpenAI and says "answer this question based on this codebase"

- OpenAI reads everything and gives back a proper answer
- You get the answer back in the response

So you could ask things like "how does user authentication work?" or "what does the search feature do?" or "is there any admin functionality?" and it will answer correctly based on the actual code stored.

How is the data stored in MongoDB?

Every file that gets processed is saved as one document in a collection called codes. Each document has 4 fields:

- fileName — the name of the file like DogsList.js
- filePath — the path of the file inside the project
- code — the full raw code of that file
- summary — the AI generated plain English summary

You can go to MongoDB Atlas, open your cluster, click Browse Collections, and you will see all these documents sitting there.

What is the current limitation?

Right now the search feature works by dumping the entire codebase into one OpenAI prompt. This works fine for small to medium repos like your petadoption project. But if someone uploads a very large project with thousands of files, the prompt will become too big and OpenAI will reject it. That is the next thing to solve but we are keeping it simple for now.

Final Summary

In short, we built a working backend server that can take any GitHub repo or uploaded code files, automatically read and understand them using AI, store everything in a database, and let you ask questions about the code in plain English. Everything was tested live on your own petadoption project and it worked successfully end to end.

WEEK 8

How We Fixed the Search Problem in AI Code Navigator

What Was the Problem?

Before the fix whenever someone asked a question like "how does login work?", the system would take every single file stored in the database, grab all their summaries, and dump everything into one big message to ChatGPT. Then ChatGPT would try to answer from that huge pile of text.

This was bad for three reasons:

- If there were 100 files, it would send all 100 summaries at once which is too much text
- ChatGPT would hit a token limit and crash or give a bad answer
- Even if it worked, the answer was vague because ChatGPT was guessing from summaries, not reading the actual code

---

What is the Fix?

The fix is called **Vector Search using Embeddings**.

The idea is simple Instead of sending everything to ChatGPT, we first find only the files that are actually relevant to the question, then send just those files.

To find the relevant files, we convert text into numbers. OpenAI has a model that reads any text and converts it into a list of 1536 numbers. These numbers represent the meaning of that text. If two pieces of text have similar meaning, their numbers will be close to each other.

What Did We Actually Change?

Step 1 – MongoDB Schema We added a new field called embedding to store the 1536 numbers for each file.

Step 2 – New Function in OpenAI Service We wrote a function called generateEmbedding that takes any text and returns 1536 numbers from OpenAI's embedding model.

Step 3 – Save Embeddings When Uploading Every time a file gets uploaded or a repo gets processed, we now generate an embedding from its summary and save it to MongoDB along with the file.

Step 4 – Vector Search Index in MongoDB Atlas We created a special index in MongoDB Atlas called vector_index. This tells MongoDB how to search through those 1536 numbers efficiently.

Step 5 – Fixed the Search Function Instead of fetching all files, the search now:

1. Converts the user's question into 1536 numbers
2. Asks MongoDB to find the top 5 files whose numbers are closest
3. Sends the actual code of only those 5 files to ChatGPT
4. ChatGPT reads the real code and gives a proper answer

Results

We tested it with the petadoption repo (55 files). When asked "how does user authentication work?", instead of sending all 55 files, the system found exactly the 5 relevant files:

- AuthContext.js
- Login.js
- Loginasadmin.js
- firebaseconfig.js
- Privateroute.js

ChatGPT then read the actual code of those 5 files and gave a detailed step by step explanation of how login, admin login, Firebase and route protection all work together.

Summary

Before the fix, search was broken for large codebases. After the fix, it finds only the relevant files and gives accurate answers based on real code, not just summaries.